

GET IT RUNNING ON ANY MACHINE

Setup in six steps

About fifteen minutes, most of it waiting for the demo to build.

YOU NEED

A paid Claude plan

Pro, Max, Team or Enterprise. **The free plan does not include Claude Code**, and this app calls Claude only through Claude Code.

YOU NEED

Python 3.9 or newer

Check with `python3 --version`. macOS 13+, Windows 10+ or a modern Linux.

YOU DON'T NEED

An API key

There is no `ANTHROPIC_API_KEY`, no Console account and no second provider. Your subscription is the only credential.

Install and sign in

STEPS 1-2

01

Install Claude Code

One command. It installs a self-updating native binary.

```
# macOS, Linux, WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex
```

Alternatives: `brew install --cask claude-code` on macOS, or `npm install -g @anthropic-ai/claude-code` with Node.js 22+. Confirm it worked with `claude --version`.

02

Sign in with your Claude account

Run **claude** in any folder. It opens a browser on first launch; sign in with the account that holds your paid plan. The terminal says **Login successful** when it is done.

```
claude
# then, inside Claude Code:
/status # confirms the account and plan in use
/login  # sign in again, or switch account
```

If the browser can't reach the callback (common over SSH or in WSL), press `c` to copy the login URL, then paste the code back into the terminal. `claude doctor` prints installation diagnostics without starting a session.

How it reaches Claude

YOUR ACCOUNT

Paid Claude plan

Pro, Max, Team or Enterprise

SIGNED IN ONCE

Claude Code CLI

Browser login, stored locally

THE APP

One call per job

Shells out to the CLI, schema-checked

IF YOU ALREADY HAVE AN API KEY SET

Claude Code prefers `ANTHROPIC_API_KEY` over your subscription. Run `unset ANTHROPIC_API_KEY` and check `/status` to confirm which credential is active.

Get the project running

STEPS 3–6

03 Get the code

```
git clone https://github.com/ayon-chatterjee/ai-procurement-analyst.git
cd ai-procurement-analyst
```

04 Install the Python dependencies

Five packages: Streamlit, pandas, jsonschema, openpyxl and Pillow.

```
pip3 install -r requirements.txt
```

05 Build the demo

This creates the database from nothing and reads five real supplier documents with the real model. **It takes about three minutes.**

```
python3 scripts/seed_demo.py --extract
```

You get one RFQ with 30 line items, five suppliers replying in five different formats, one who never replies, one revision, one self-contradiction, one verified certificate and four claimed ones. Drop `--extract` to register the responses unread and press **Run extraction** in the app instead. Add `--clean` to remove every other RFQ first.

06 Run it

```
python3 -m streamlit run app.py
```

Open `http://localhost:8501` and press **Open** on **Start here — the worked example**. If port 8501 is taken, add `--server.port 8502`.

Check it really works

OPTIONAL

The test suite needs no network, no API key and no model calls, so it is the fastest way to prove a clone is healthy. It should report **565 tests**, all passing, in under a minute.

```
python3 -m unittest discover -s tests -t .
```

Where everything lives

ALL LOCAL

PATH	WHAT'S IN IT
<code>data/rfq_copilot.db</code>	Every RFQ, supplier response, extracted quote, analyst query and award. Created on first run.
<code>data/uploads/</code>	Supplier documents you add through the app.
<code>data/fx_cache.json</code>	Exchange rates fetched from a public endpoint, cached for six hours.
<code>fixtures/suppliers/</code>	The committed demo documents the seed script reads. These <i>are</i> in the repository.

The whole `data/` folder is gitignored and never transfers between machines — it is real sourcing data. `seed_demo.py` rebuilds a complete demo from the fixtures, so nothing of value is lost by deleting it.

LOOKS RIGHT WHEN The sidebar reads **Claude Code • signed in**. If it says Claude Code isn't authenticated, the app is fine and the sign-in is not — go back to step 2. That is the single most common cause of a confusing first five minutes.

Or let Claude Code do it

COPY & PASTE

Once you are signed in (steps 1–2), open a terminal in the project folder, run **claude**, and paste any of these. It can do the rest of the setup for you.

SET IT UP

“Read README.md and HANDOVER.md first. Then set this project up on my machine: install the Python dependencies, seed the demo database with real extraction, and start the app. Tell me the URL when it's running. If anything fails, stop and tell me exactly what to fix rather than working around it.”

PROVE IT WORKS

“Run `python3 -m unittest discover -s tests -t .` and report the total number of tests and any failures. Don't change any code.”

FIX A BROKEN SETUP

“The app reports that Claude Code isn't authenticated / extraction keeps failing. Diagnose it: check `claude doctor`, check whether `ANTHROPIC_API_KEY` is set, and check the `RFQ_*` settings in `rfq_copilot/config.py`. Tell me the one thing to change.”

RESET BEFORE A DEMO

“Re-seed the demo so the supplier response dates are current again: run `python3 scripts/seed_demo.py --extract --clean`, then confirm the worked example opens and the expiring-quote warning is live rather than expired.”

UNDERSTAND THE CODE

“Explain how a supplier's PDF becomes one row in the comparison table. Name the modules it passes through in order, and point out exactly where the system decides what it is and isn't willing to assert.”

CHANGE A DEFAULT

“I want extraction to read eight responses at a time instead of four. Read `rfq_copilot/config.py`, tell me the exact environment variable, and show me how to set it before launching the app.”

RUN THE LIVE CHECKS

“Run `scripts/analyst_smoke.py` and `scripts/award_smoke.py`. Both work on a copy of the database and never write to it. Report what passed and anything that looked wrong.”

If something goes wrong

WHAT YOU SEE	WHAT TO DO
“Claude Code isn't authenticated”	Run <code>claude</code> and sign in; check with <code>/status</code> . Everything else can be fine and the app will still look broken.
<code>command not found: claude</code>	The install didn't finish, or <code>~/local/bin</code> isn't on your PATH. Re-run the installer, then <code>claude --version</code> .
A usage-limit message on a Team plan	Check <code>/status</code> reports the team subscription. A stale personal entitlement is refused even when the team account has capacity.
It's using an API key, not your plan	<code>unset ANTHROPIC_API_KEY</code> , then re-check <code>/status</code> .
Extraction feels slow	Normal. 30–90 seconds per supplier response, four at a time, so five suppliers take about 90 seconds.
A model call timed out	Press Try again — a retry always gets the full time allowance, so it does something different from the attempt that failed.
Quotes show as expired	The demo data is dated relative to the seed run. Re-run step 5 with <code>--clean</code> .
Port 8501 already in use	Run with <code>--server.port 8502</code> .

NOTHING LEAVES YOUR MACHINE

No email is ever sent, no supplier is contacted, and the order handoff is a file you download. The database, the uploads and the cached exchange rates all live in a local `data/` folder.